

Programare Orientată Pe Obiecte

Curs 5

Pachete

Moștenire

Cuvântul cheie super

Construirea obiectelor când avem moștenire

Variabile și metode statice în contextul moștenirii

Octombrie 2025

Pachete

Organizarea claselor înrudite în pachete

- Pachet (package): set de clase înrudite
- Pentru a pune o clasă într-un pachet, trebuie scrisă o astfel de linie **package numePachet;** ca primă instrucțiune în fișierul sursă care conține clasa
- Numele pachetului constă din unul sau mai mulți identificatori separați prin puncte
- Spre exemplu, pentru a pune clasa **Database** într-un pachet numit **oop.examples**, fișierul **Database.java** trebuie să înceapă astfel:

```
package oop.examples;  
public class Database  
{  
    ...  
}
```

- Pachetul implicit nu are nume, deci nu are o specificare **package**

Pachete importante în Java și funcționalitățile lor

Pachet	Funcționalitate
<code>java.lang</code>	Conține clasele de bază pentru tipuri de date, manipularea stringurilor, obiecte wrapper pentru primitive, și clase de gestionare a erorilor și excepțiilor. Acesta este importat implicit.
<code>java.util</code>	Furnizează clase pentru structuri de date (liste, seturi, cozi, hărți), clase pentru manipularea datei și timpului, și alte utilitare generale.
<code>java.io</code>	Permite manipularea intrărilor și ieșirilor (I/O), lucrul cu fișiere, citirea și scrierea fluxurilor de date și serializarea obiectelor.
<code>java.nio</code>	Oferă funcționalități pentru procesarea I/O non-blocantă și acces direct la buffer-ele de memorie, pentru manipularea eficientă a datelor.

Pachete importante în Java și funcționalitățile lor

(Cont.)

Pachet	Funcționalitate
<code>java.net</code>	Conține clase pentru programarea rețelei, inclusiv suport pentru conexiuni TCP/IP, lucru cu URL-uri și protocoale de rețea.
<code>java.sql</code>	Asigură suport pentru accesul la baze de date relaționale și utilizarea SQL, incluzând conectivitate prin JDBC.
<code>javax.swing</code>	Include clase pentru construirea interfeței grafice de utilizator (GUI), cu componente cum ar fi ferestre, butoane și meniuri.

Importul pachetelor

- Se poate folosi întotdeauna o clasă fără import

```
java.util.Scanner s = new java.util.Scanner(System.in);
```

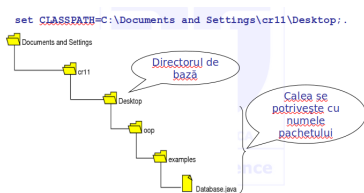
- Dar e greoi să folosim nume calificate complet
- import** ne permite să folosim nume mai scurte pentru clase

```
import java.util.Scanner;  
...  
Scanner in = new Scanner(System.in)
```

- Putem importa toate clasele dintr-un pachet: **import java.util.*;**
- Nu este nevoie să importăm java.lang
- Nu este nevoie să importăm alte clase din același pachet

Nume de pachete și determinarea locului unde se află clasele

- Folosiți pachete pentru a evita conflictele de nume (două clase diferite având același nume - **Timer** - situate în două pachete diferite): **java.util.Timer** vs. **javax.swing.Timer**
- Numele de pachete trebuie să fie neambigue
- Numele căii trebuie să se potrivească cu numele pachetului:
oop/examples/Database.java
- Calea spre clase conține directoarele de bază care pot conține directoare de pachet



Cum se construiește un pachet

- 1 Puneți o linie cu numele pachetului la începutul fiecărei clase.

```
package pachetDulciuri;
public class Ciocolata {
    ...
}

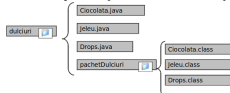
package pachetDulciuri;
public class Jeleu {
    ...
}

package pachetDulciuri;
public class Drops {
    ...
}
```

- 2 Stocați fișierele Java din pachet într-un director comun



- 3 Compilați toate fișierele.



- 4 Importați pachetul după nevoi:

```
import dulciuri.pachetDulciuri.*;
public class ConsumatorDulciuri { ... }
```


Cum să refolosim codul:

Compoziție

```
class Carte{
public String toString() {
return "Carte ["+titlu +
    ", autor="+autor+"]";
}
public Carte(String titlu,
String autor) {
    this.titlu = titlu;
    this.autor = autor;
}
String titlu;
String autor;
}
```

```
class Librarie{
private int nbCarti = 0;
private Carte[] listaCarti;
public Librarie(int n) {
    listaCarti = new Carte[n];
}
public void adaugaCarte(Carte c) {
    listaCarti[nbCarti] = c;
    nbCarti++;
}
public void afiseazaCarti() {
    for (int i = 0; i < nbCarti; i++)
        System.out.println(listaCarti[i]);
} }
```

```
public class LibrarieDemo {
public static void main(String[] args) {
    Carte c1 = new Carte("EffectiveJ Java", "Joshua Bloch");
    Carte c2 = new Carte("Thinking in Java", "Bruce Eckel");
    Librarie lib = new Librarie(10);
    lib.adaugaCarte(c1); lib.adaugaCarte(c2); lib.afiseazaCarti();
} }
```

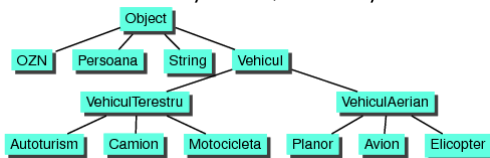

Moștenirea

(Cont.)

Moștenirea modelează relații de tipul "este o(un)"

- Un obiect "este un" alt obiect dacă se poate comporta în același fel
- Moștenirea folosește **asemănările** și **deosebirile** pentru a modela grupuri de obiecte înrudite

Unde există moștenire, există și o **ierarhie de moștenire** a claselor.



- Vehicul este numit superclasă sau clasă de bază sau clasă părinte
- VehiculTerestru este numit subclasă sau clasă derivată sau clasă fiică
- Oricare clasă poate fi de ambele feluri în același timp: D.e., VehiculTerestru este superclasă pentru Camion și subclasă pentru Vehicul

Moștenirea

(Cont.)

- În Java fiecare clasă extinde clasa Object fie direct, fie indirect
- O clasă are în mod automat toate variabilele instanță și metodele clasei de bază și poate avea și metode suplimentare și/sau variabile instanță
- Moștenirea este avantajoasă deoarece permite să se refolosească codul, fără a fi nevoie să fie copiat în definițiile claselor derivate
- În Java se poate moșteni de la o **singură** superclasă
- Nu există limite pentru adâncimea sau lățimea ierarhiei de clase

Exemplu: ierarhia unor forme geometrice

```
class Shape {
    protected String color;
    public Shape(String color) { this.color = color;    }
    // Method to get area (Overridden in subclasses)
    protected double getArea() { return 0.0;    }
    // Overloaded method to display info
    public void displayInfo() {
        System.out.println("Shape color: " + color);
    }
    public void displayInfo(String additionalInfo) { // Method
        Overloading
        System.out.println("Shape color: " + color + ", " +
            additionalInfo);
    }
    // Tip returnat covariant
    protected Shape createSimilarShape() {
        return new Shape(this.color);
    }
}
```

Exemplu: ierarhia unor forme geometrice

(Cont.)

```
class Circle extends Shape {  
    private double radius;  
    public Circle(String color, double radius) {  
        super(color);  
        this.radius = radius;  
    }  
    // Overriding getArea method  
    @Override  
    public double getArea() { return Math.PI * radius * radius; }  
  
    // Overloading displayInfo to include radius  
    public void displayInfo(double radius) {  
        System.out.println("Circle color: " + color + ", radius: " +  
            radius);  
    }  
  
    // Tip returnat covariant in metoda suprascrisa  
    @Override  
    public Circle createSimilarShape() {  
        return new Circle(this.color, this.radius);  
    }  
}
```

Exemplu: ierarhia unor forme geometrice

(Cont.3)

```
// Subclass Rectangle
class Rectangle extends Shape {
    private double length; private double width;
    public Rectangle(String color, double length, double width) {
        super(color);
        this.length = length;
        this.width = width;
    }
    // Overriding getArea method
    @Override
    public double getArea() {return length * width;}
    // Overloading displayInfo to include dimensions
    public void displayInfo(double length, double width) {
        System.out.println("Rectangle color: " + color + ", length: "
            + length + ", width: " + width);
    }
    // Tip returnat covariant in metoda suprascrisa
    @Override
    public Rectangle createSimilarShape() {
        return new Rectangle(this.color, this.length, this.width);
    }
}
```

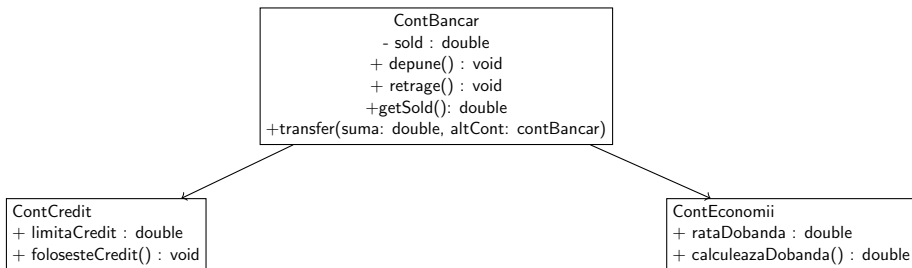
Exemplu: ierarhia unor forme geometrice

(Cont.4)

```
public class Main {  
    public static void main(String[] args) {  
        Shape genericShape = new Shape("Gray");  
        genericShape.displayInfo();  
        System.out.println("Generic shape area: " + genericShape.  
            getArea());  
        Circle circle = new Circle("Red", 5.0);  
        circle.displayInfo();  
        circle.displayInfo(circle.radius); // Overloaded displayInfo  
        System.out.println("Circle area: " + circle.getArea());  
        Rectangle rectangle = new Rectangle("Blue", 4.0, 6.0);  
        rectangle.displayInfo();  
        rectangle.displayInfo(rectangle.length, rectangle.width); //  
            Overloaded displayInfo  
        System.out.println("Rectangle area: " + rectangle.getArea());  
        // Demonstrate covariant return types  
        Shape similarCircle = circle.createSimilarShape();  
        Shape similarRectangle = rectangle.createSimilarShape();  
        System.out.println("Created similar shapes: " +  
            similarCircle.getClass().getSimpleName() + ", " +  
            similarRectangle.getClass().getSimpleName());  
    } }
```

Exemplu: ierarhia unor conturi bancare

- ContBancar este clasa de bază, care definește metodele comune pentru un cont bancar.
- ContCredit și ContEconomii sunt clase derivate, fiecare având metode și atribute specifice.



Exemplu: ierarhia unor conturi bancare

- Toate conturile bancare suportă metoda **getSold()** - obține soldul contului
- Toate conturile bancare suportă metodele **depune** și **retrage**, dar implementările diferă
- Contul de credit are nevoie de o metodă pentru deducerea taxelor de prelucrare – **deductFees**;
- Contul de economii are nevoie de o metodă pentru adăugarea dobânzii - **calculeazăDobanda()**

Clase derivate

Cum un cont de economii este un cont bancar, el este definit ca o clasă derivată a clasei BankAccount.

- O clasă derivată se definește prin adăugarea de variabile și/sau metode la o clasă existentă
- Fraza `extends BaseClass` trebuie adăugată în definiția clasei derivate:

```
public class ContEconomii extends ContBancar
```

- Sintaxa pentru moștenire:

```
class NumeSubclasa extends NumeSuperclasa  
{  
    metode  
    campuri de instanta  
}
```

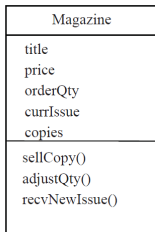
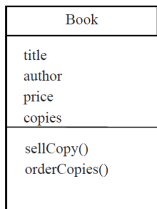
Clase derivate (subclase)

- O clasă derivată, numită și subclasă, este definită pornind de la o altă clasă definită deja, numită clasă de bază sau superclasă, prin adăugarea (și/sau modificarea) de metode, variabile instanță și de variabile statice
- Clasa derivată moștenește toate metodele, toate variabilele instanță, precum și toate variabilele statice din clasa de bază
- Clasa derivată poate adăuga variabile instanță, variabile statice și/sau metode
- Definițiile variabilelor și metodelor moștenite nu apar în clasa derivată
- Codul este reutilizat fără a fi nevoie să fie copiat explicit, cu excepția cazului în care creatorul clasei derivate nu redefinește una sau mai multe dintre metodele clasei

Clase derivate

Exemplu

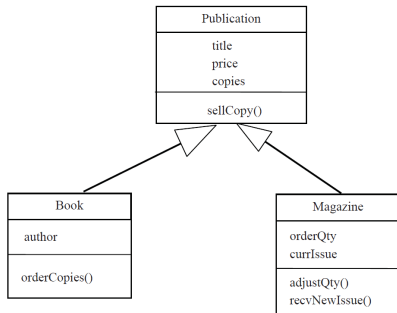
Exemplu fără moștenire:



Ce au în comun cele 2 clase ?

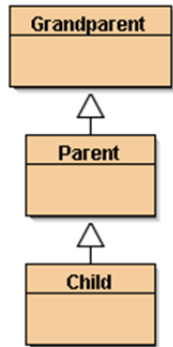
Ce diferă între cele 2 clase ?

Exemplu cu moștenire:



Clase părinți și clase copii

- O clasă de bază este numită adesea clasă părinte
- Clasa derivată se mai numește și clasă fiică (copil)
- Aceste relații sunt adesea extinse astfel că o clasă este părintele unui părinte al unei alte clase și se numește clasă strămoș
- Dacă clasa **Grandparent** este un strămoș al clasei **Child**, atunci clasa **Child** poate fi numită clasă descendentă a clasei **Grandparent**



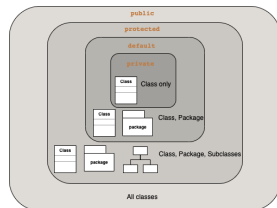
Variabile instanță

Ca șablon general, subclasele:

- Moștenesc capabilitățile **public** (metode)
- Moștenesc proprietățile **private** (variabile instanță) dar nu au acces la ele
- Moștenesc variabilele **protected** și le pot accesa

O variabilă declarată **protected** de o superclasă devine **parte a moștenirii**:

- Variabila devine disponibilă pentru subclase, care o pot accesa **ca și cum ar fi proprie**
- Spre deosebire de aceasta, dacă o variabilă instanță este declarată **private** într-o superclasă, subclasele nu vor avea acces la ea
- Superclasa poate totuși oferi acces protejat la variabilele instanță private via metode accesoare și mutatoare



Modifier	Class	Package	Subclass	World
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
no modifier*	✓	✓	✗	✗
private	✓	✗	✗	✗

Variabile instanță protected față de variabile instanță private

Cum putem decide între private și protected?

- Folosiți private dacă doriți ca o variabilă instanță să fie încapsulată de către superclasă d.e., ușile, ferestrele, bujiile unei mașini
- Folosiți protected dacă doriți ca variabila instanță să fie accesibilă subclaselor pentru a o modifica (și nu doriți să faceți variabila mai general accesibilă prin metode acceseoare/mutatoare) d.e., motorul unei mașini

protected, Exemplu

```
public class Vehicle1 {
    protected String make;
    protected String model;
    public Vehicle1() { make = ""; model = ""; }
    public String toString() {
        return "Make: " + make + " Model: " + model;
    }
    public String getMake() { return make; }
    public String getModel() { return model; }
}

public class Car1 extends Vehicle1 {
    private double price;
    public Car1() { price = 0.0; }
    public String toString() {
        return "Make: " + make + " Model: " + model
            + " Price: " + price;
    }
    public double getPrice() { return price; }
}
```

private, Exemplu

```
public class Vehicle1 {
    private String make;
    private String model;
    public Vehicle1() { make = ""; model = ""; }
    public String toString() {
        return "Make: " + make + " Model: " + model;
    }
    public String getMake(){ return make;}
    public String getModel() { return model;}
}

public class Car1 extends Vehicle1 {
    private double price;
    public Car1() { price = 0.0; }
    public String toString() {
        return "Make: " + #getMake()# + " Model: " +
@getModel()@ + " Price: " + price;
    }
    public double getPrice(){ return price; }
}
```

Suprascrierea unei definiții de metodă

Deși o clasă derivată moștenește metode din clasa de bază, ea poate să le modifice - să le suprascrîe dacă este necesar

- Pentru a suprascrîe o definiție de metodă, se pune pur și simplu o definiție nouă în definiția clasei, exact ca pentru orice altă metodă adăugată clasei derivate
- De obicei, tipul returnat nu poate fi schimbat la suprascrîerea unei metode
- Totuși, dacă tipul este un tip clasă, atunci tipul returnat poate fi schimbat la acela al oricărei clase descendente al tipului returnat
- Acest lucru se cunoaște sub numele de **tip returnat covariant**

Tip returnat covariant

Exemplu

```
class Document {
    String content;
    Document(String content) { this.content = content; }
    Document cloneDocument() { return new Document(this.content); }
    void printType() {
        System.out.println("Doc. type: General Document");
    }
}

class PDFDocument extends Document {
    PDFDocument(String content) { super(content); }
    PDFDocument cloneDocument() {
        return new PDFDocument(this.content); // tip Covariant }
    void printType() {System.out.println("Doc. type: PDF Document");}
}

public class Main {
    public static void main(String[] args) {
        Document genericDoc = new Document("General Content");
        Document copiedGenericDoc = genericDoc.cloneDocument();
        copiedGenericDoc.printType();
        PDFDocument pdfDoc = new PDFDocument("PDF Content");
        PDFDocument copiedPdfDoc = pdfDoc.cloneDocument();
        copiedPdfDoc.printType(); // Output: Document type: PDF
    }
}
```

Schimbarea permisiunii de acces a unei metode suprascrise

- Permiseunea de acces a unei metode suprascrise poate fi schimbata de la private in clasa de baza la public sau **alt acces mai permisiv in clasa derivata**
- Totusi, permisiunea de acces a unei metode suprascrise nu poate fi modificata de la public in clasa de baza la o permisiune de acces mai restrictiva in clasa derivata
- Adica, putem relaxa permisiunile de acces intr-o clasa derivata, nu o putem restrange

Suprascrierea unei definiții de metodă

Exemplu

```
class LibraryItem {
    protected String title;
    public LibraryItem(String title) { this.title = title; }
    // Default access in superclass
    protected void checkout() {
        System.out.println("Check out a library item: " + title);
    } }

class Book extends LibraryItem {
    public Book(String title) { super(title); }
    // Override with a more accessible modifier: protected -> public
    @Override
    public void checkout() {
        System.out.println("Check out book:" + title);
    } }

class Magazine extends LibraryItem {
    public Magazine(String title) { super(title); }
    // Override with a more accessible modifier: protected -> public
    @Override
    public void checkout() {
        System.out.println("Check out magazine: " + title);
    } }
```

Suprascrierea unei definiții de metodă

Exemplu

```
public class Main {  
    public static void main(String[] args) {  
        LibraryItem genericItem = new LibraryItem("Generic Item");  
        Book book = new Book("Java Programming");  
        Magazine magazine = new Magazine("Tech Monthly");  
        // Demonstrating access and overridden methods  
        genericItem.checkout();    // Accesses protected method in  
                                   superclass  
        book.checkout();           // Accesses public overridden method  
                                   in Book  
        magazine.checkout();       // Accesses public overridden method  
                                   in Magazine  
    }  
}
```

Capcană: Suprascriere față de supraîncărcare

Nu confundați suprascrierea (overriding) unei metode într-o clasă derivată cu supraîncărcarea (overloading) numelui unei metode;

- Când o metodă este suprascrisă, noua definiție de metodă dată în clasa derivată are exact același număr și tipuri de parametri ca în clasa de bază.
- Când o metodă dintr-o clasă derivată are o semnătură diferită în comparație cu metoda din clasa de bază, atunci avem de-a face cu supraîncărcarea.
- Observați că atunci când clasa derivată suprascrie metoda originală, ea totuși moștenește și metoda originală din clasa de bază.

Capcană: Suprascriere față de supraîncărcare

Suprascriere (overriding)

- Argumentele trebuie să fie la fel;
- Tipul returnat să fie compatibil (tip returnat covariant – subclasa)
- Metoda nu poate fi mai puțin accesibilă

Supraîncărcarea: (overloading)

- Argumentele trebuie să fie diferite;
- Tipul returnat poate fi diferit
- Nu se poate schimba DOAR tipul returnat. Trebuie schimbată și lista argumentelor.
- Nivelele de acces pot fi variate oricum.

Modificatorul **final**

- Dacă se pune modificatorul final în fața definiției unei metode, atunci metoda respectivă nu poate fi suprascrisă într-o clasă derivată
- Dacă modificatorul final este pus în fața definiției unei clase, atunci clasa respectivă nu mai poate fi folosită pe post de clasă de bază pentru a deriva alte clase

37 / 58

Constructorul **super**

O clasă derivată folosește un constructor al clasei de bază pentru a inițializa toate datele moștenite din clasa de bază

- Pentru a invoca un constructor al clasei de bază, se folosește o sintaxă specială:

```
public DerivedClass(int p1, int p2, double p3)
{
    super(p1, p2);
    instanceVariable = p3;
}
```

- În exemplul de mai sus, `super(p1, p2);` este un apel al constructorului clasei de bază

Accesul la o metodă redefinită din clasa de bază

În definiția unei metode dintr-o clasă derivată, versiunea suprascrisă a unei metode a clasei de bază poate totuși fi invocată:

- Pur și simplu prefixați numele metodei cu `super` și un punct:

```
public String toString()
{
    return (super.toString() + "$" + interestRate);
}
```

- Cu toate acestea, la folosirea unui obiect al clasei derivate în afara definiției clasei, nu există nici o cale de invocare a versiunii unei metode suprascrise din clasa sa de bază

Rolu Cuvântului Cheie **super** în Java

- ❶ La apelul explicit al constructorului din clasa părinte: `super(...)`;
 - Este obligatoriu dacă părintele nu are constructor fără argumente.
- ❷ Apelarea metodelor părintelui: `super.metodă(...)`
 - Permite accesul la implementarea originală din clasa părinte, chiar dacă este suprascrisă în subclasă.
- ❸ Accesarea câmpurilor părintelui ascunse: `super.câmp`
 - Se folosește când subclasa definește un câmp cu același nume și dorim să accesăm valoarea câmpului din clasa părinte.

Exemplu clasă: ierarhia de bază

```
class Shape {  
    protected String name;  
    protected String color = "black";  
    public Shape(String name) { this.name = name; }  
    public Shape(String name, String color) {  
        this(name);  
        this.color = color;  
    }  
    public double area() {return 0.0}  
    @Override public String toString() {  
        return name + "(" + color + ")";  
    }  
}
```

Reguli pentru `super ( . . . )`

- Dacă este prezent, `super ( . . . )` trebuie să fie **prima** instrucțiune din constructor.
- `super ()` implicit există doar dacă `Shape` are constructor fără argumente.
- Cu `super()` se poate apele orice constructor din clasa parinte.

Exemplu: `Cerc` — apel constructor părinte

```
class Cerc extends Shape {  
    private double r;  
    public Cerc(double r) {  
        super("Cerc");    // apeleaza Shape(String)  
        this.r = r;  
    }  
    public Cerc(double r, String color) {  
        super("Cerc", color); // apeleaza Shape(String,String)  
        this.r = r;  
    }  
    //metoda suprascrisa  
    @Override  
    public double area() { return Math.PI * r * r; }  
}
```

Exemplu: Dreptunghi — constructori

```
class Dreptunghi extends Shape {
    private double w, h;
    public Dreptunghi(double w, double h) {
        super("Dreptunghi");
        this.w = w; this.h = h;
    }
    public Dreptunghi(double w, double h, String color) {
        super("Dreptunghi", color);
        this.w = w; this.h = h;
    }
}

@Override
public double area() { return w * h; }
}

class DemoOrder {
    public static void main(String[] a){
        new Cerc(2.0);
        System.out.println("---");
        new Cerc(3.0, "red");
    }
}
```

super.metodă(...)

```
class Cerc extends Shape {  
    private final double r;  
    public Cerc(double r, String color){  
        super("Cerc", color); this.r = r;  
    }  
    @Override  
    public String toString() {  
        return super.toString() + " r=" + r; // porneste de la parinte  
    }  
}  
class Dreptunghi extends Shape {  
    private final double w,h;  
    public Dreptunghi(double w,double h,String color){  
        super("Dreptunghi", color);  
        this.w=w; this.h=h;  
    }  
    @Override  
    public String toString(){  
        return super.toString() + " w=" + w + " h=" + h;  
    }  
}
```

super.câmp

```
class Shape {
    protected String color = "black";
}

class Cerc extends Shape {
    protected String color = "blue"; // ascunde campul parintelui
    public void demoCulori(){
        System.out.println(color);           // "blue" (Cerc)
        System.out.println(super.color);     // "black" (Shape)
    }
}

class CercColorat extends Cerc {
    public CercColorat(double r){ super(r, "green"); }
    @Override public String toString(){
        return "CercColorat(" + super.toString() + " "
            + "cu culoarea=" + super.color;
    }
}

class Demo2 {
    public static void main(String[] args){
        var c = new CercColorat(5);
        System.out.println(c);
    }
}
```

super vs. this

- `this` → instanța curentă (metode suprascrise, câmpuri din clasa obiectului).
- `super` → referă membrii *clasei părinte* (constructori, metode moștenite, câmpuri ascunse).
- Nu poate fi folosit în context `static`.
- Nu există `super.super`.

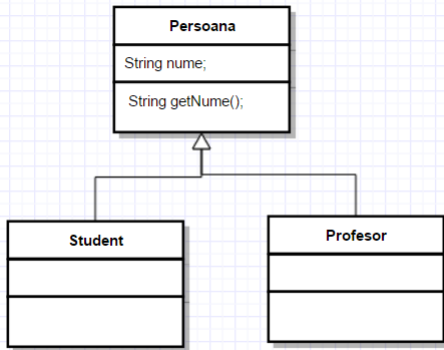
Capcane frecvente

- Fără constructor fără argumente în părinte ⇒ `super(...)` devine obligatoriu.
- `super(...)` trebuie să fie prima linie a constructorului.
- Câmpurile sunt ascunse, nu suprascrise.
- Acces doar la membri `public/protected` (sau de pachet, dacă e cazul).

Temă: modificați ierarhia Shape, Cerc, Dreptunghi

- 1 Adăugați `borderWidth (protected)` în `Shape`; setați valoarea lui prin constructor în `Cerc`.
- 2 În `Dreptunghi.toString()` inserați și informații despre arie.
- 3 Creați `CercNegru` care ascunde `color` cu `"black"` și compară `color` vs. `super.color`.

Construirea obiectelor în JAVA



```
public class Persoana{
    private String nume;
    public String getNume(){
        return nume;
    }
}

class Student extends Persoana{

}

class Profesor extends Persoana{

}
```


Construirea obiectelor în JAVA

(Cont.)

Cum interpretează compilatorul Java codul din acest exemplu?

Regula 1:

Dacă o clasă nu extinde o alta clasă, atunci compilatorul inserează implicit: `extends Object`

```
public class Persoana{
    private String nume;
    //...
```



```
public class Persoana extends Object{
    private String nume;
    //...
```

Construirea obiectelor în JAVA

(Cont.)

Cum interpretează compilatorul Java codul din acest exemplu?

Regula 2:

dacă într-o clasă nu este definit nici un constructor, compilatorul creează implicit constructorul fără parametri

```
public class Persoana{  
    private String nume;  
    //...  
}
```



```
public class Persoana extends Object{  
    private String nume;  
    Persoana(){  
    }  
    //...  
}
```

Construirea obiectelor în JAVA

(Cont.)

Cum interpretează compilatorul Java codul din acest exemplu?

Regula 3:

prima linie din interiorul constructorului trebuie să fie:

- fie apelul unui alt constructor: **this(<params>)**
- fie apelul unui constructor din superclasă: **super(<params>)**

Altfel, compilatorul apelează implicit constructorul superclasei fără parametri `super()`

```
public class Persoana{  
    private String nume;  
    //...  
}
```



```
public class Persoana extends Object{  
    private String nume;  
    Persoana(){  
        super();  
    }  
}
```

Membrii statici

Metode și variabile statice în contextul moștenirii:

- Membrii statici (metode sau variabile) aparțin clasei, nu instanțelor ei.
- Se accesează de obicei prin numele clasei:

```
Shape.count++;  
Shape.printInfo();
```

- **super** și **this** sunt referințe la instanțe, iar membrii statici aparțin clasei, nu obiectului.
- Codul de mai jos nu dă eroare de compilare:

```
super.staticMethod(); // Eroare de compilare
this.staticMethod(); //Eroare de compilare
```

Moștenirea variabilelor statice

- O subclasă moștenește câmpurile statice ale clasei părinte, dar nu le copiază; există o singură copie comună.
- Dacă le redeclarăm cu același nume, nu le „suprascrim”, ci le ascundem (shadowing).

```
class Shape {
    static int count = 0;
    public Shape() { count++; }
}
class Cerc extends Shape {
    static int count = 100; // ascunde count-ul din Shape
}
public class Demo {
    public static void main(String[] args) {
        new Shape();
        new Cerc();
        System.out.println(Shape.count); // 1
        System.out.println(Cerc.count); // 100
    }
}
```

Observatie: Shape.count si Cerc.count sunt variabile diferite.

Moștenirea metodelor statice

- Metodele statice nu sunt polimorfice — adică nu pot fi suprascrise (overridden), ci doar ascunse (hidden).
- Tipul variabilei de referință (nu tipul obiectului real) determină metoda apelată.

```
class Shape {  
    static void whoAmI() { System.out.println("Sunt Shape"); }  
}  
class Cerc extends Shape {  
    static void whoAmI() { // ascunde metoda parintelui  
        System.out.println("Sunt Cerc");    }  
}  
public class Demo {  
    public static void main(String[] args) {  
        Shape s = new Cerc();  
        s.whoAmI();    // -> "Sunt Shape"  
        Cerc.whoAmI();    // -> "Sunt Cerc"  
    }  
}
```

Observație: chiar dacă **s** referă un **Cerc**, apelul **s.whoAmI()** se leagă la compilare, nu la execuție. *static* $\neq$ *polimorfism*

Exercițiu:

- Construiti codul JAVA folosind urmatoarea ierahie de clase.
- Adaugati variabilele instantă si metodele necesare.

